



FINAL REPORT

Minion Me Marketplace

Sandra Wahu Ngugi 18/03030 Unit: BISF 4500

3/25/26

Abstract

Minion Me is a web-based marketplace built to connect people who need help with daily tasks (Clients) to service providers, whom the system calls Minions. The project was inspired by the need to digitize the informal task-sourcing process in the local economy, which is currently unorganized and often lacks a sense of trust. The platform automates this by providing a secure environment for account registration, email verification, and task management.

The system was developed as a full-stack application using React and Vite for the frontend, Express.js for the backend logic, and PostgreSQL for data management. Security was a major focus, so I implemented JWT-based authentication and role-based access control to ensure users can only access features relevant to their roles (Client, Minion, or Admin). Key features include a task-posting workflow, a payment recording system, and a rating mechanism for accountability. The final prototype shows that a lightweight, locally-focused platform can effectively organize task management and make the service exchange process more transparent for everyone involved.

Contents

Abstract	1
Chapter One.....	4
1.0 Background.....	4
2.0 Problem Statement.....	4
3.0 Objectives	5
4.0 Significance of Project.....	5
Chapter Two	7
2.1 Literature Review	7
2.2 Systems Requirements Specifications	8
2.2.1 Functional Requirements	8
2.2.2 Non-Functional Requirements	9
2.2.3 Hardware and Software Requirements	10
Chapter Three	11
3.1 Data Collection Methods	11
3.2 Systems Design Specifications	11
3.2.1 Architectural Design	11
3.2.2 Major Modules.....	12
3.2.3 Input Design.....	14
3.2.4 Output Design	14
3.2.5 Database Design Specification.....	15
3.3 Test Plan	15
3.4 Implementation Plan.....	17
Chapter Four.....	18
4.1 Graphical User Interface Test Results	18
4.2 Database test cases.....	19

4.3 System output test cases	20
Chapter Five	22
5.1 Conclusions	22
5.2. Contributions	22
5.3. Recommendations	23
References	24
Appendices	25
Appendix I–III: Appendices in Work progress document.....	Error! Bookmark not defined.
Appendix IV: User Training Manual.....	Error! Bookmark not defined.
Appendix V. Sample System Code	25
Appendix VI. Sample	26

Chapter One

1.0 Background

In many local settings, finding a service provider still relies heavily on informal methods like phone calls, personal referrals, and social media posts. While these traditional channels are common, they lack an organized way to compare different workers, track the progress of a job, or maintain a clear record of payments. This lack of accountability often leads to conflict between the client and the provider, making it difficult for clients to trust the process and limiting the earning potential for skilled workers who do not have a formal platform to display their reliability.

Minion Me was developed as a digital solution to centralize this process. The system allows a client to post a specific task and receive applications from available "Minions." By allowing clients to review worker profiles and assign the most suitable candidate, the platform brings transparency to the informal economy. From a technical standpoint, the project utilizes a client-server architecture. I chose React for the frontend to handle route-based navigation and a responsive user interface. The backend logic is managed through Express.js REST endpoints, with PostgreSQL used for persistent data storage. This stack was selected because it is modular and can be deployed on affordable infrastructure, making it a sustainable choice for a prototype that could eventually be scaled into a production-ready marketplace.

2.0 Problem Statement

Clients looking for help with daily tasks often face uncertainty when trying to find reliable service providers. Currently, there is no easy way to verify if a provider is available or trustworthy before hiring them. On the other side, skilled individuals who offer task-based services struggle to find a consistent platform where they can display their skills, discover new work, or build a professional reputation through a history of completed jobs.

The existing informal methods of hiring are difficult to monitor and often lead to poor record keeping. This creates a lack of transparency regarding the status of a task, whether a payment has been processed fairly, and the overall quality of the service provided.

This project addresses the lack of a centralized, role-based digital platform that can handle the posting, matching, and management of small to medium tasks. Without such a system, the process of hiring and paying for local services remains unorganized and lacks the accountability needed for a secure and transparent service exchange.

3.0 Objectives

The primary objective of this project was to design and implement a web-based service marketplace that effectively connects clients with service providers (minions).

To achieve this, the following specific objectives were established:

- ✓ To develop secure authentication and authorization workflows, including registration and login for three distinct user roles: clients, minions, and administrators.
- ✓ To create a task management system that allows clients to post requirements, review applications from available workers, assign tasks, and monitor their progress.
- ✓ To enable service providers (minions) to build professional profiles, browse available local gigs, apply for tasks, and manage their accepted jobs.
- ✓ To implement a payment and rating framework that ensures accountability and builds trust between parties once a service is completed.
- ✓ To design an administrative dashboard that provides centralized oversight for managing users, tracking tasks, and verifying payment records.
- ✓ To develop and test a functional prototype to ensure the system meets usability standards, handles data consistently, and performs its core functions correctly.

4.0 Significance of Project

The development of Minion Me provides several benefits to different participants involved in the local task-based economy:

- I. For Clients: The platform offers a structured and reliable way to post jobs, review applicants, and track work status. This moves the process away from unpredictable informal referrals and gives clients more control over who they hire and how the work is managed.
- II. For Minions (Service Providers): It creates a searchable source of opportunities and increases their professional visibility. By completing tasks and receiving ratings, workers can build a digital reputation that helps them secure future gigs, which is often difficult in the informal sector.
- III. For Administrators: The system provides centralized visibility into user activities, job postings, and payment records. This is essential for platform governance, ensuring that disputes can be resolved and that the marketplace remains secure for all users.
- IV. For Researchers and Students: This project serves as a practical demonstration of how to apply core computer science concepts such as, web engineering, security controls, relational database design, and software testing, to solve a real-world local problem.
- V. For Future Platform Owners: The prototype provides a scalable foundation that can be further enhanced. It establishes a baseline architecture that could eventually support more advanced features like live chat, automated escrow services, and detailed user analytics.

Chapter Two

2.1 Literature Review

Digital marketplaces have fundamentally changed how temporary services are sourced by matching supply and demand through searchable profiles and structured listings. In these platforms, trust is usually established through a combination of user ratings, transparent transaction records, and strict access control. Minion Me adopts these core concepts but adapts them to a local context, focusing on everyday tasks such as deliveries, errands, domestic help, and short-duration technical installations.

A review of existing marketplace models shows that successful systems generally follow a specific functional pattern: user onboarding, profile management, task publishing, and a clear application or bidding flow. These are typically followed by task assignment, payment tracking, and a final service review. For this project, I have incorporated these essential functions into a simplified prototype to demonstrate a complete, end-to-end service cycle that is both functional and easy to navigate.

The technical foundation of the project is built on established web engineering practices. I chose React for the frontend because its component-based architecture makes it easier to build a responsive user interface. On the backend, Express.js was used for its lightweight API routing and middleware capabilities, while PostgreSQL was selected as the relational database to ensure data integrity through constraints and transactional queries. These technologies were chosen not only for their performance but also because they are well-supported by extensive documentation, which is crucial for building a reliable, role-based system.

Since marketplace platforms handle sensitive information like user identities and payment records, security is a major priority. To address this, the project implements several foundational controls, including password hashing, JWT-based authentication, and server-side validation. I also included role-based authorization and rate limiting to ensure that the system remains secure and that users can only access the features permitted by their specific roles.

2.2 Systems Requirements Specifications

2.2.1 Functional Requirements

The Minion Me system is designed around several core modules that define how different users interact with the platform. The functional requirements for each module are as follows:

- ✓ **Authentication and User Access:** The system must provide a secure way for users to join the platform. This includes registration, email verification to ensure valid accounts, secure login, password reset functionality, and a logout mechanism.
- ✓ **Role-Based Access Control:** To maintain security and proper workflow, the system must strictly differentiate access rights. This ensures that Clients, Minions, and Administrators can only access the specific dashboards and features relevant to their roles.
- ✓ **Task Creation and Management:** Clients must be able to create new task listings by providing details such as descriptions and requirements. They also need the ability to update, view, and manage their posted tasks throughout the project lifecycle.
- ✓ **Gig Discovery and Application:** The platform must allow Minions to browse a list of open tasks. They should be able to filter through opportunities and submit applications for work that matches their skills.
- ✓ **Assignment Workflow:** Once applications are received, Clients must be able to review the profiles of interested Minions and formally accept a preferred candidate to start the task.
- ✓ **Task Progress Tracking:** After a task is assigned, the system must allow the Minion to update the status of the job and mark specific milestones or final completion to keep the Client informed.
- ✓ **Payment Recording:** The system is required to track the financial side of the transaction. This includes recording when a payment is initiated and updating the status once the workflow is finalized.
- ✓ **Feedback and Ratings:** To ensure accountability, Clients must have the ability to rate a Minion's performance after the successful completion of a task.

- ✓ **System Administration:** The backend must provide an administrative dashboard where authorized staff can monitor all users, tasks, and payment records, as well as view general platform statistics for oversight.

2.2.2 Non-Functional Requirements

Beyond the core features, the Minion Me platform must meet specific performance and quality standards to ensure it is reliable and user-friendly. These non-functional requirements include:

- ✓ **Usability and User Experience:** The interface is designed to be simple and visually clear. Since the platform targets a wide range of users, it must be easily navigable even by individuals with only basic web literacy.
- ✓ **Performance and Responsiveness:** The system should handle common actions—such as logging in, loading the task list, or posting a new job—within a reasonable response time. This ensures the application remains functional even on standard internet connections typical of the local deployment environment.
- ✓ **Security Controls:** Protecting user data is a high priority. The system must implement password hashing for all accounts, ensure that authenticated routes are protected from unauthorized access, and perform strict server-side validation on all user inputs to prevent common web vulnerabilities.
- ✓ **System Availability:** The application is built to be flexible and should be deployable on either common cloud platforms or local servers. This ensures the prototype is always available for demonstrations, testing, and potential pilot use.
- ✓ **Maintainability and Code Structure:** I have used a modular approach for both the frontend pages and backend routes. By keeping these components independent, specific features can be updated or debugged without affecting the rest of the system.
- ✓ **Scalability and Future Growth:** The database schema and API route structures are designed to be scalable. This foundation ensures that the platform can support a growing number of users, job postings, and transaction records as the marketplace expands.

2.2.3 Hardware and Software Requirements

To develop and deploy the Minion Me platform, a specific set of hardware and software resources were utilized. These requirements ensure that the system is built on a stable environment and that the chosen tech stack supports the project's security and functional goals.

2.2.3.1 Hardware Requirements

For the development phase, the project required a laptop or desktop computer with at least 8 GB of RAM and reliable internet access. This hardware was necessary to simultaneously run the frontend development server, the backend API, and the PostgreSQL database instance without performance bottlenecks.

2.2.3.2 Software and Development Stack

The software environment was selected to be cross-platform, making it suitable for development on Windows, Linux, or macOS. The specific technologies used include:

- ✓ **Frontend Stack:** The user interface is built using React with Vite for a fast development build cycle. React Router is used for client-side navigation, while Axios handles API requests. For styling, I used Tailwind-style UI utilities to maintain a clean, responsive design.
- ✓ **Backend Stack:** The server logic is powered by Node.js and Express.js. To handle security and communication, I integrated JWT for sessions, Argon2 for secure password hashing, and Nodemailer for email notifications. Express Validator and Express Rate Limit were also implemented to protect the API from malicious inputs and brute-force attempts.
- ✓ **Database Management:** PostgreSQL was used as the primary relational database to manage complex relationships between users, tasks, and payments while ensuring data persistence and integrity.

Chapter Three

3.1 Data Collection Methods

The requirements for Minion Me were gathered through a practical, problem-oriented approach focused on understanding the local service economy. The goal was to identify specific gaps in the current way tasks are managed and translate those into technical features.

The research process involved several key stages:

- I. **Observation of Current Practices:** I began by observing how people in local settings currently find short-term service providers. This highlighted major pain points, such as delayed responses from workers, the uncertainty of worker availability, and a general lack of accountability once a job is assigned.
- II. **Comparative Review of Existing Platforms:** I conducted a review of established digital marketplaces to identify the "standard" features that users expect. This informed the core workflows of the system, such as how profiles should be displayed and how the bidding/application process should function.
- III. **Technical Document Review:** To ensure the system was built on a secure and modern foundation, I performed an extensive review of official documentation for the selected tech stack (Node.js, React, and PostgreSQL). This helped in selecting the best libraries for security, such as Argon2 and JWT.
- IV. **Iterative Prototyping and Refinement:** Throughout the development phase, I used an iterative approach. By simulating actual user journeys for both the Client and the Minion, I was able to refine the requirements based on what felt most intuitive and functionally correct during testing.

3.2 Systems Design Specifications

3.2.1 Architectural Design

The Minion Me platform is built using a three-tier architecture. This structure was chosen because it separates the user interface, the business logic, and the data management, making the system more modular, secure, and easier to maintain.

The three layers are defined as follows:

- I. **The Presentation Layer (Frontend):** This is the top level of the application, responsible for everything the user sees and interacts with. I developed this as a React single-page application to ensure a smooth, modern user experience. It handles all client-side navigation through React Router, manages forms for task posting and registration, and provides dedicated dashboards for the different user roles. Axios is used here to communicate with the backend API.

Technologies used: ***React, Vite, React Router, Axios.***

- II. **The Application Layer (Backend):** This layer acts as the mind of the system, sitting between the frontend and the database. Built with Express.js, it exposes several route modules that handle the core business logic. This includes managing authentication through JWT and Argon2, validating task applications, processing payment records, and sending notifications via Nodemailer. This layer ensures that all incoming requests are authorized before any data is modified.

Technologies used: ***Node.js, Express.js, JWT, Argon2, Nodemailer.***

- III. **The Data Layer (Database):** The bottom tier is the PostgreSQL relational database. It is responsible for the persistent storage of all system data, including user profiles, task listings, payment statuses, and ratings. I chose a relational database to ensure data integrity through the use of foreign key constraints and indexes, which are vital for tracking the relationships between clients, minions, and their assigned tasks.

Technologies used: ***PostgreSQL.***

3.2.2 Major Modules

Frontend Modules

Routing and Access Control: Managed primarily in `src/App.jsx` and `ProtectedRoute.jsx`, this module maps out the public and private pages of the application. I implemented role-based

navigation to ensure that a Minion cannot access a Client's dashboard and that unauthorized users are redirected to the login page.

Authentication Context: The `src/context/AuthContext.jsx` file is the heart of the frontend's security. It bootstraps the user session and manages the authenticated state across the entire application, ensuring the user remains logged in as they navigate between different tasks.

Backend API Modules

Authentication Routes: Located in `backend/routes/auth.js`, this module handles the heavy lifting of user security, including registration, login, email verification, and password resets. It also manages user lookup queries to verify credentials against the database.

Task Management: The `backend/routes/tasks.js` file contains the logic for the "CRUD" operations (Create, Read, Update, Delete) of the marketplace. This is where clients post new jobs and where the system retrieves the list of open gigs for minions to browse.

Assignment and Workflow: Managed in `backend/routes/assignments.js`, this module controls the logic for task applications and assignments. It tracks when a minion applies for a job and allows the client to formally assign them, while also handling progress updates until the task is marked as complete.

Payment Integration: The `backend/routes/payments.js` module is designed to handle payment initiation and record retrieval. It is structured to support the callback handling necessary for a mobile-money workflow, ensuring that every task has a corresponding payment status.

Administrative Oversight: The `backend/routes/admin.js` file provides gathered system statistics. This module allows an administrator to get a high-level view of platform activity, including the total number of users, active tasks, and successful payments.

Database Layer

Database Initializer: The `backend/db.js` file is responsible for the connection setup between the Express server and PostgreSQL. It handles the core table definitions, constraints, and indexes, ensuring the database schema is correctly patched and ready for use when the server starts.

3.2.3 Input Design

The Minion Me platform relies on a series of structured inputs to ensure that the data flowing through the system is accurate and complete. I designed the following forms to capture the essential details needed for the marketplace to function:

- ✓ **User Registration Forms:** These are used to onboard new members. The system captures the user's full name, email address, phone number, and password. I also included fields for location and role-specific details (Client vs. Minion) to ensure users are directed to the correct dashboard after signing up.
- ✓ **Task Posting Form:** When a Client needs help, they use this form to define the work. It captures a clear task title, a detailed description, the location where the task is needed, the category (e.g., delivery, domestic help), and the offered budget.
- ✓ **Task Application Form:** For Minions looking for work, this input captures their formal interest in a specific gig. It allows the Minion to submit any necessary task-application information that the Client might need to review before making an assignment.
- ✓ **Payment Initiation Form:** To handle the financial workflow, this form captures the task reference and the total amount to be paid.
- ✓ **Feedback and Rating Form:** Once a task is marked as complete, the Client is presented with an input to provide a score based on the Minion's performance. This input is critical for building the reputation system mentioned in the project objectives.

3.2.4 Output Design

- Public and role-specific dashboards summarizing platform activities.
- Task lists for clients and gig listings for minions.
- Application status and assignment feedback.
- Payment status messages and payment history views.
- Administrative summary views for users, tasks, and payments.

3.2.5 Database Design Specification

The Minion Me system utilizes a relational database model to manage the complex interactions between different users and their tasks. I chose PostgreSQL for the data layer because it allows for strict enforcement of data integrity through foreign key constraints, unique indexes, and structured schemas.

The database is designed around four core entities that represent the primary workflows of the marketplace:

Entity Relationship Overview

- ✓ **Users:** This is the central table for the platform. It stores essential profile information including `first_name`, `last_name`, email, phone, location, and a hashed password. I included a role field to differentiate between Clients, Minions, and Admins, and an `email_verified` Boolean to support the secure onboarding process.
- ✓ **Tasks:** This entity handles the lifecycle of a job. It captures the title, description, location, category, and budget. Each task is linked to a `client_id` (Foreign Key), and a status field is used to track whether a task is "Open," "Assigned," or "Completed."
- ✓ **Payments:** For the prototype version of this system, the payments table stores manual confirmation records. It captures the `task_id`, `client_id`, `minion_id`, and the amount. It also includes a `payment_confirmed_at` timestamp to provide an audit trail for completed work, ensuring that both parties have a record of the financial transaction.
- ✓ **Minion Ratings:** To maintain the accountability mentioned in the project objectives, this table stores a rating (score) linked to a specific `task_id`. This allows the system to calculate a Minion's reputation over time based on feedback from different Clients.

3.3 Test Plan

The testing phase for Minion Me focused on validating the graphical user interface, route protection, database consistency, and system outputs. I used a manual functional testing approach, creating specific test accounts for each user role (Client, Minion, and Administrator) to simulate

real-world usage. My strategy included both positive testing (verifying the system works as intended) and negative testing (ensuring the system correctly handles invalid inputs and unauthorized access).

Core Test Areas and Objectives

- ✓ **User Authentication and Access:** The objective was to ensure secure account access. I tested the full onboarding cycle, including registration and email verification. To test the security of the login system, I verified that valid credentials granted access while invalid attempts (such as incorrect passwords or non-existent emails) were correctly rejected. I also validated the password reset workflow to ensure users could regain account access securely.
- ✓ **Role-Based Protection and Security:** Given the importance of security in this project, I performed strict checks on route protection. I verified that a user logged in as a Minion was blocked from accessing Client-specific pages and that ordinary users could not bypass security to reach the Administrative dashboard. This confirmed that the JWT-based authorization was correctly enforcing role boundaries.
- ✓ **End-to-End Task Workflow:** I tested the complete lifecycle of a task to ensure the system handled the transitions correctly. This involved a Client posting a task, a Minion browsing and applying for that task, and the Client successfully reviewing and accepting the application. Finally, I verified that the Minion could update the task status through to completion.
- ✓ **Payment and Transaction Recording:** Testing focused on the accuracy of the payment recording workflow. I simulated client confirmation of a manual payment and inspected the system's ability to record the transaction and update the payment status accordingly.
- ✓ **Post-Completion Ratings:** To verify the accountability mechanism, I tested the rating submission process. I confirmed that a Client could only submit a rating once a task was marked as completed and the payment conditions were met, preventing premature or fraudulent reviews.
- ✓ **Administrative Oversight:** Finally, I tested the Admin dashboard to ensure all oversight functions were operational. This included loading the full user, task, and payment lists, as

well as verifying that the summary statistics accurately reflected the data stored in the PostgreSQL database.

3.4 Implementation Plan

The Minion Me project was implemented using an incremental approach. By breaking the development into specific phases, I was able to reduce the risk of major bugs and ensure that each milestone was fully tested before moving on to the next. This sequence reflects the practical path I took to build the prototype.

Phase 1: Requirements Analysis

I began by analyzing the core problem and defining the project's primary objectives. This involved identifying the specific workflows for both Clients and Minions to create a finalized feature list and project scope.

Phase 2: System Design

During this phase, I selected the three-tier architecture and planned the API routes. I also designed the PostgreSQL schema and drafted the interface layouts to ensure the frontend and backend would integrate seamlessly.

Phase 3: Frontend Development

I focused on building the user interface using React and Vite. This included creating all registration forms, task posting pages, and role-specific dashboards. I also implemented the Protected Routes logic to manage user access.

Phase 4: Backend Development

The core logic was built using Node.js and Express.js. I developed the API routes for authentication, task management, and payments. I also wrote the middleware for JWT validation and the database queries needed to interact with the data layer.

Phase 5: System Integration

This was a critical phase where I connected the React frontend to the Express backend. I tested the end-to-end workflows, such as a Client posting a task and a Minion seeing it in real-time, to ensure the two layers communicated correctly.

Phase 6: Final Testing and Debugging

Once the system was integrated, I performed comprehensive testing on the GUI and the database. I verified all outputs and fixed any bugs identified during the manual functional testing of the different user roles.

Phase 7: Final Documentation

The final stage involved compiling all project materials. This included writing the final project report, creating a user manual for the platform, and organizing the technical appendices for submission.

Chapter Four

4.1 Graphical User Interface Test Results

- I. Platform Access and Branding: I began by testing the basic accessibility of the application. The home page loaded successfully, correctly displaying the Minion Me branding and the main navigation menu. All public links were functional, leading to the intended registration and login pages.
- II. Role-Based Authentication: To test the login system, I used valid credentials for both Client and Minion accounts. In both cases, the system correctly identified the user role and redirected them to their specific dashboard. This confirmed that the AuthContext and session management logic were working as designed.
- III. Security and Route Protection: A critical part of the GUI testing was ensuring that users could not bypass role-based restrictions. I attempted to access a Client dashboard using a Minion account and tried to reach the Admin panel as an ordinary user. In every attempt, the Protected Route logic successfully blocked unauthorized access and redirected the user, validating the security of the frontend architecture.

- IV. Task Management and Gig Discovery: I verified the core marketplace features by testing the Post Task form. As a Client, I was able to submit a new task with all required fields, receiving a successful confirmation from the server. On the Minion side, I tested the Find Gigs view, which correctly rendered the list of available tasks, allowing for easy browsing and application.
- V. User Profile and Discovery: I also tested the Client's ability to "Browse Minions." The interface successfully loaded the available service provider profiles, confirming that the frontend was correctly fetching and displaying relational data from the PostgreSQL database.
- VI. Administrative Oversight: Finally, I logged in as an Administrator to test the oversight dashboard. The system successfully rendered the overview metrics and management sections, displaying real-time statistics for users, tasks, and payments across the platform.

4.2 Database test cases

- I. User Record Persistence: I tested the system's ability to insert and retrieve new user records. The database successfully stored the `first_name`, `last_name`, and `email` while correctly assigning the role (Client or Minion). This confirmed that the users table was correctly handling persistent storage for all platform members.
- II. Relational Integrity for Tasks: To verify the relationship between users and jobs, I tested the insertion of a new task. The database correctly linked the task to the specific `client_id` through a foreign key constraint and initialized the status field, ensuring that every task has a clear owner and a starting state.
- III. Payment Constraint Validation: I conducted negative testing on the payments table to ensure financial data integrity. When I attempted to insert a negative payment amount or an invalid task reference, the database rejected the transaction. This confirmed that the constraints I implemented were effectively preventing invalid data from entering the system.
- IV. Rating Uniqueness and Accountability: To prevent fraud, I tested the unique constraint on the `minion_ratings` table. When I tried to submit a second rating for the same task by

the same client, the database blocked the entry. This ensures that each task can only be rated once, maintaining the accuracy of the Minion's reputation score.

- V. State Management Constraints: I verified that the status field in the tasks table only accepted pre-defined values (such as "Open," "Assigned," or "Completed"). Any attempt to insert an unexpected status was rejected, ensuring that the task lifecycle remains consistent across the platform.
- VI. Indexing and Retrieval Efficiency: Finally, I verified that the database was optimized for search. During the initialization phase, I confirmed that the relevant indexes were created for the token and status fields. This ensures that as the number of users and tasks grows, the system can still perform lookups efficiently without performance degradation.

4.3 System output test cases

Session Initialization and User State: I tested the login output to ensure that a JWT-based session was correctly established. Upon a successful login, the system successfully initialized the user session and loaded the correct role-specific navigation. This confirmed that the frontend was receiving the necessary authentication tokens to manage the user's state.

- I. Email Verification Workflow: To test the onboarding security, I initiated an email verification request. The system successfully generated a unique verification link and logged the request. This output confirms that the backend is ready to handle the Nodemailer integration for verifying new accounts.
- II. Secure Password Reset: A critical security test was performed on the password reset request. I verified that the system returns a generic success response regardless of whether the email exists in the database. This output design is intentional, as it prevents account enumeration by not exposing whether a specific email has an account on the platform.
- III. Task Application and Visibility: I tested the output of the task application process. Once a Minion applied for a gig, the system correctly recorded the application and updated the related views for both the Client and the Minion. This confirmed that the PostgreSQL triggers and frontend state updates were synchronized.

- IV. Assignment State Transitions: When a Client accepted a Minion, I verified that the task state changed to "Accepted" or "Assigned" across the platform. The system correctly reflected this assignment in the job tracking screens, providing immediate feedback that the workflow had progressed to the next stage.
- V. Payment Record Retrieval: I tested the system's ability to fetch and display financial data. For a completed task, the system successfully returned the payment details, showing the amount and confirmation status. This output ensures that users have a transparent view of their transaction history.
- VI. Rating and Reputation Workflow: Finally, I verified the output of the rating system. After a task was marked as finished, the rating submission was successfully stored and linked to both the task and the Minion. The workflow completed without errors, ensuring that the feedback loop for accountability was fully functional.

Overall, the comprehensive testing phase demonstrates that the Minion Me prototype successfully achieves the intended functional workflows. From the initial account creation and email verification through to task completion and user reviews, the system consistently performed as expected.

Chapter Five

5.1 Conclusions

The development of Minion Me successfully resulted in a functional prototype of a web-based marketplace designed to bridge the gap between clients and task workers in the Kenyan gig economy. The project demonstrates that a role-based architecture can effectively digitize the entire service lifecycle, from task posting and worker search, to assignment, progress tracking, payment recording, and final reviews, within a single, unified platform.

The selected technology stack (React, Node.js, Express, and PostgreSQL) proved to be highly effective for rapid development and modular integration. The use of a three-tier architecture allowed for a clean separation of concerns, ensuring that the user interface remained responsive while the backend handled complex business logic and security enforcement.

While the prototype is fully suitable for academic demonstration and controlled pilot use, the implementation process also highlighted critical areas that require further strengthening before any full-scale production deployment. Specifically, observations made during development and testing pointed toward the need for more robust session handling, stricter data exposure controls, and comprehensive environment hardening.

These findings are particularly valuable from an Information Security perspective. They illustrate the vital intersection where software engineering meets security, proving that building a functional system is only the first step; ensuring that the system is resilient against real-world threats is a continuous and essential part of the development lifecycle.

5.2. Contributions

- ✓ Designed and implemented a full-stack marketplace prototype with three distinct user roles.
- ✓ Integrated key service-platform features including task posting, worker application, assignment, payment, and rating.
- ✓ Applied practical security controls such as hashing, JWT-based route protection, validation, and rate limiting.
- ✓ Produced a modular codebase that can be extended into a more robust startup or institutional platform.

- ✓ Demonstrated how local payment-context thinking can be incorporated into a service marketplace workflow.

5.3. Recommendations

- Migrate authentication from browser-stored tokens to more secure HttpOnly cookie-based sessions where feasible.
- Tighten CORS, environment validation, and production secret management before deployment.
- Add stronger worker identity verification, dispute handling, and in-platform messaging.
- Introduce search filters, analytics, audit logs, and notification preferences for better usability.
- Enhance the payment module with stricter production-safe workflows and reconciliation checks by using M-Pesa integrations.

References

Auth0. (2024). JSON Web Token (JWT) architecture and security best practices. Auth0 Docs. <https://auth0.com/docs/secure/tokens/json-web-tokens>

Barden, W. (2023). React and Express: Building full-stack web applications with the MERN stack (2nd ed.). O'Reilly Media.

Chodorow, K. (2020). MongoDB: The definitive guide: Powerful and scalable data storage. O'Reilly Media. (Note: Even though you use PostgreSQL, this is a standard text for comparing relational vs. non-relational data layers).

Express.js. (2026). Express: Fast, unopinionated, minimalist web framework for Node.js (Version 4.x). <https://expressjs.com/>

Fielding, R. T., & Taylor, R. N. (2000). Architectural styles and the design of network-based software architectures. University of California, Irvine. (The seminal paper for RESTful API design).

Lodderstedt, T., McGloin, M., & Hunt, P. (2024). OAuth 2.0 threat model and security considerations (RFC 6819). IETF Trust. <https://datatracker.ietf.org/doc/html/rfc6819>

OWASP Foundation. (2025). OWASP Top 10:2025 – The ten most critical web application security risks. <https://owasp.org/www-project-top-ten/>

PostgreSQL Global Development Group. (2026). PostgreSQL 16.0 Documentation. <https://www.postgresql.org/docs/16/index.html>

React Team. (2026). React: The library for web and native user interfaces. Meta Open Source. <https://react.dev/>

Stallings, W. (2022). Computer security: Principles and practice (5th ed.). Pearson Education. (Excellent for justifying your use of password hashing and encryption).

Vite.dev. (2026). Vite: Next generation frontend tooling. <https://vitejs.dev/guide/>

Appendices

Appendix I-III: Work Progress Documents

(Refer to project submission folder).

Appendix IV: User Training Manual

(Attached separately).

Appendix V. Sample System Code

Frontend route protection (excerpt from src/App.jsx)

```
<Route path="/client-dashboard" element={
  <ProtectedRoute allowedRoles={['client']}>
    <ClientDashboard />
  </ProtectedRoute>
} />
```

Auth session bootstrap (excerpt from src/context/AuthContext.jsx)

```
const bootstrapSession = useCallback(async () => {
  const token = localStorage.getItem('token');
  if (!token) { resetAuthState(); return; }
  const response = await authAPI.getCurrentUser({ skipAuthRedirect: true });
  setUser(response.data);
  setIsAuthenticated(true);
}, [resetAuthState]);
```

Backend route registration (excerpt from backend/index.js)

```
app.use("/auth", authRoutes);
app.use("/tasks", taskRoutes);
app.use("/assignments", assignmentRoutes);
```

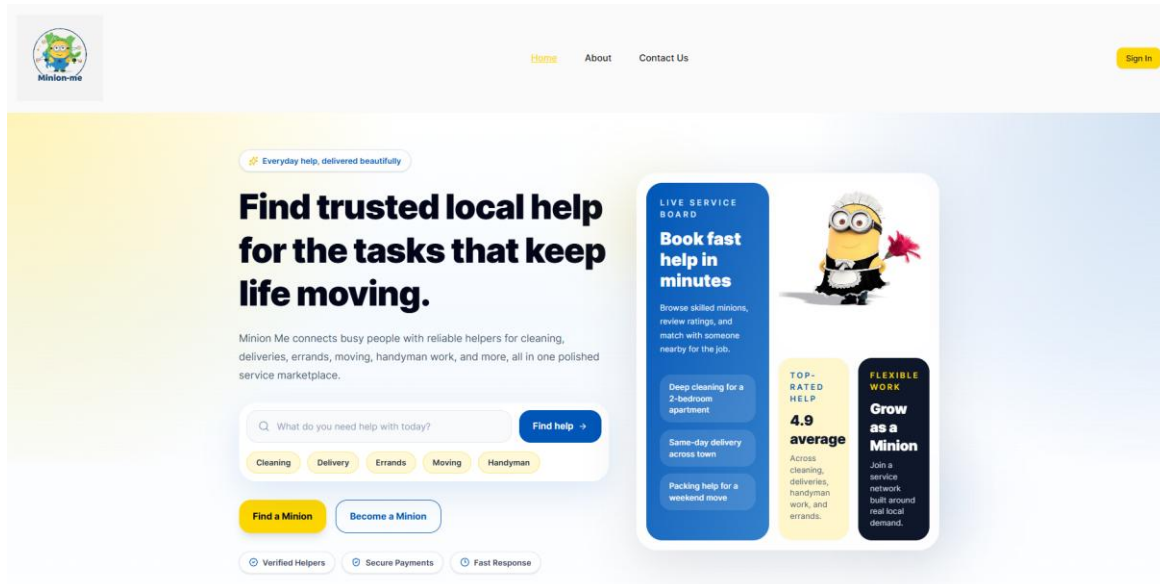
```
app.use("/payments", paymentRoutes);
app.use("/ratings", ratingRoutes);
app.use("/admin", adminRoutes);
```

Database support for payments and ratings (excerpt from backend/db.js)

```
CREATE TABLE IF NOT EXISTS payments (
  id SERIAL PRIMARY KEY,
  task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  client_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  amount NUMERIC(12, 2) NOT NULL CHECK (amount > 0),
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
);
```

Appendix VI. Sample (System Screenshots)

Home page



About page



[Home](#)[About](#)[Contact Us](#)

Sign In

About Minion Me

Minion Me connects people who need help with people who are ready to work. Simple tasks. Real people. Fair pay. No nonsense.

Our Mission

We believe meaningful work doesn't have to be complicated. Minion Me exists to make it easy for anyone to get help with everyday tasks — and just as easy for skilled, reliable people to earn from their time and talent.

Why We Built This

Too often, small jobs fall through the cracks. They're too big to ignore and too small for traditional services. Minion Me bridges that gap, turning "I wish someone could help" into "done."

How It Works

Post a Task

Describe what you need done, set your budget, and publish your task in minutes.

Get Matched

Skilled minions nearby browse tasks and apply based on their expertise.

Get It Done

Choose the right person, get the task completed, and pay securely.

Our Values

Trust, fairness, and simplicity guide every decision we make. We're building Minion Me to empower communities, not complicate them.

Contact page

[Home](#)[About](#)[Contact Us](#)

Contact Us

Questions, feedback, or need help getting started? We're here to make things easier.

Get in Touch

Whether you're posting your first task or signing up as a minion, our support team is ready to help. We aim to respond clearly, quickly, and like actual humans.

Email: support@minionme.com

Hours: Monday – Friday, 9am – 6pm

Location: Remote-first

Name

Email

Message

Send Message

Login

Welcome Back

Log in to get help or manage your minion life

Email

Password

Log In

[Forgot client password?](#) [Forgot minion password?](#)

Don't have an account? [Sign up](#)

Register

Find a Minion

Become a Minion

Register as a Client

Get Help

Find a reliable minion to get things done.

First Name *

Jane

Last Name *

Doe

Email Address *

jane@example.com

Phone Number

+254 7XX XXX XXX

Location

Nairobi, Kenya

Password *

Create Account

Already have an account? [Log in](#)

Register as a Minion

Become a Minion

Turn your skills into opportunities. Help people. Get paid.

First Name *

Kevin

Last Name *

Otieno

Phone Number

+254 7XX XXX XXX

Email Address *

minion@example.com

Location *

Nairobi, Kenya

What skills can you offer? *

Cleaning, delivery, repairs, moving, errands...

Availability *

Select availability

Previous experience (optional)

Briefly describe your experience

Create Password *

.....

Apply to Become a Minion

Already a minion? [Log in](#)

Verify Email

EMAIL VERIFICATION

Verify your account

Open the verification link from your email, or request a new one below.

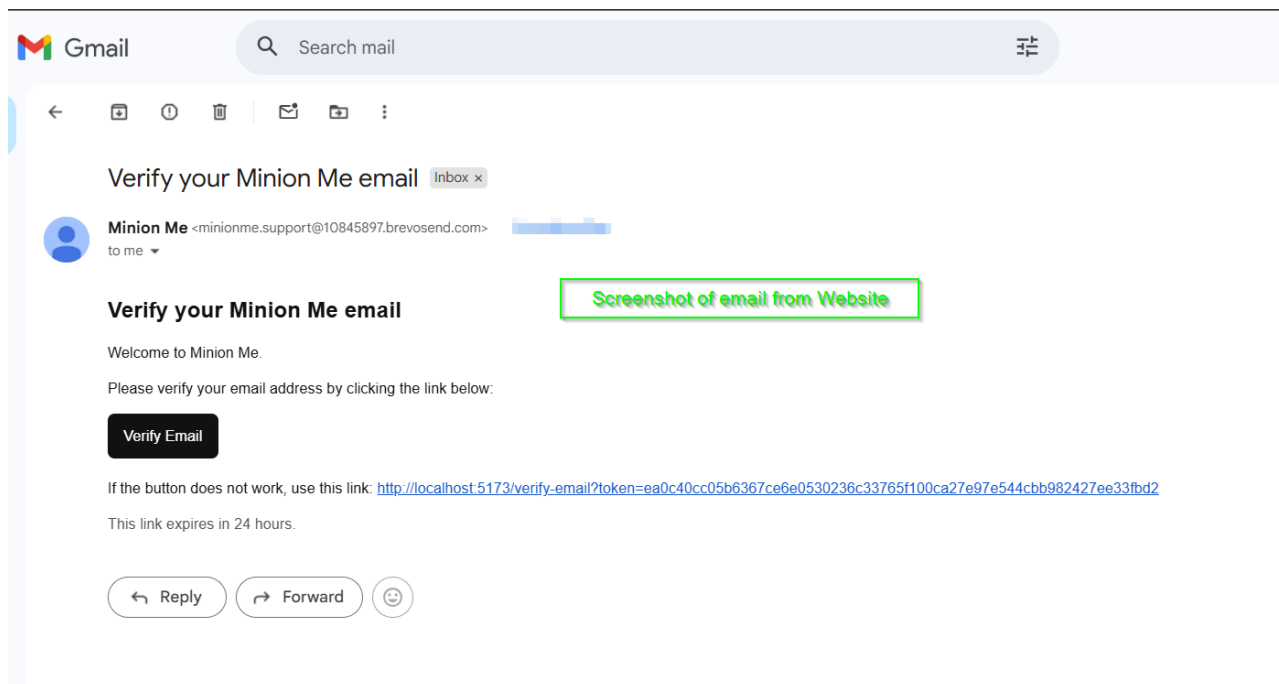
Email address

@gmail.com

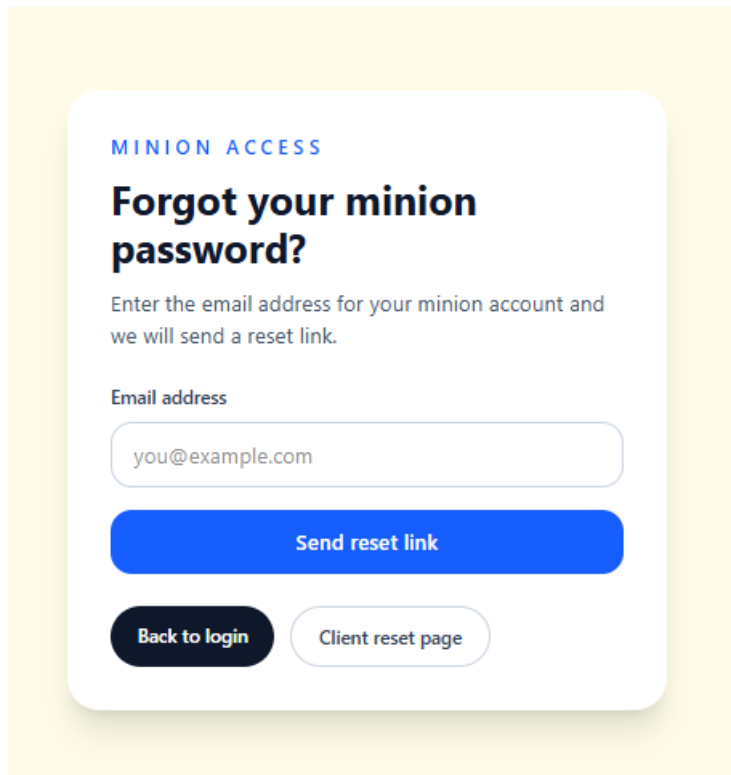
Resend verification email

Go to login Create client account

Create minion account



Forgot Password



MINION ACCESS

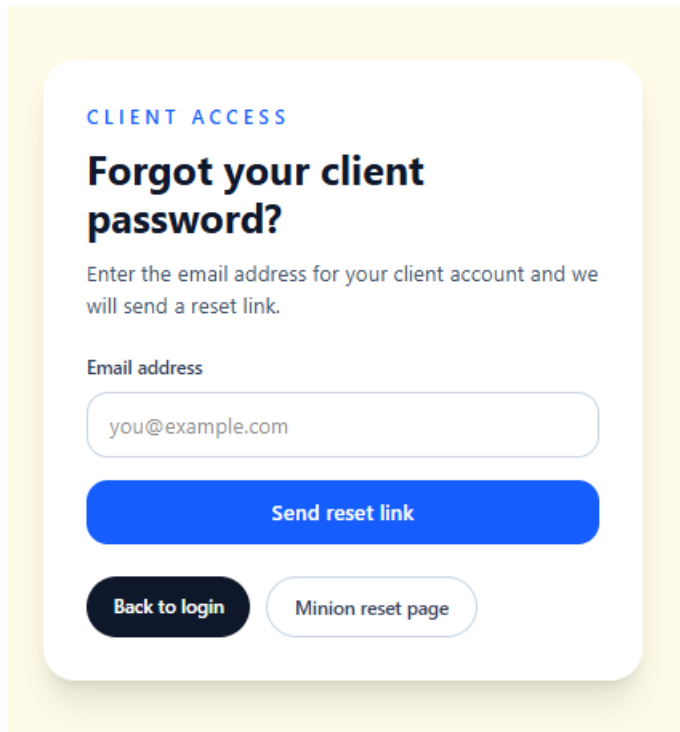
Forgot your minion password?

Enter the email address for your minion account and we will send a reset link.

Email address

Send reset link

Back to login Client reset page



CLIENT ACCESS

Forgot your client password?

Enter the email address for your client account and we will send a reset link.

Email address

Send reset link

Back to login Minion reset page

Reset Password pages

PASSWORD RESET

Choose a new password

Set a strong password for your Minion Me account.
This works for both client and minion accounts.

New password

Confirm password

Reset password

Go to login

Request another link

Client Dashboard

CLIENT DASHBOARD



Welcome back, Passion!

Keep track of your tasks, discover trusted minions, and turn your to-do list into completed work.



Logout



YOUR PROFILE

Passion Berry

Keep your contact details current so minions can coordinate quickly and complete tasks smoothly.

CLIENT

Edit Profile

EMAIL

passionberry78@gmail.com

PHONE

+254705692476

LOCATION

Nairobi

DASHBOARD SNAPSHOT

Ready to get something done?

New clients look best when the dashboard stays helpful even before the first task. Start with a clear request and Minion Me will do the rest.

Post a detailed task brief

Compare helpers by ratings and specialty

Track progress from request to payout

Post your first task →

QUICK ACTIONS

Everything you need to move a task forward

These action cards are designed to help you start fast, stay organized, and find the right help without friction.



Post a New Task

Create a detailed request and start receiving interest from available minions.



Manage My Tasks

Track active work, review progress, and stay on top of client actions.



Browse Minions

Explore skilled helpers by specialty, ratings, and location before you hire.

Post a New Task

Create a detailed request and start receiving interest from available minions.

Open →

Manage My Tasks

Track active work, review progress, and stay on top of client actions.

Open →

Browse Minions

Explore skilled helpers by specialty, ratings, and location before you hire.

Open →

ACTIVE TASKS

0

No live tasks yet. Post your first request to start getting matched.

COMPLETED

0

Finished work will appear here once your first task is completed.

TOTAL SPENT

KSh 0

Your client spend summary will update as payments are recorded.

RECENT ACTIVITY

Task updates and helpful nudges

A lively feed keeps the dashboard useful even before your task history fills up.

View notifications

No recent task updates yet

Once you post tasks and receive activity, updates will appear here in a clean timeline.

Tip: strong task briefs get better matches

Include budget, location, timing, and exact expectations to attract reliable minions faster.

Your notifications feed stays connected

Application alerts, payment updates, and task events are all available from the

TASKS OVERVIEW

A simple status snapshot

Once you start posting, these status groups help you scan progress at a glance.

PENDING

0

Waiting for minions to apply

ASSIGNED

0

A minion has been chosen

IN PROGRESS

0

Work is actively underway

COMPLETED

0

Finished tasks ready for review

CANCELLED

Post Task

34

Post a Task

Tell us what you need help with and find the perfect minion for the job.

Task Title *

e.g., Deep clean my 2-bedroom apartment

Task Description *

Provide details about what needs to be done, any specific requirements, tools needed, etc.

Be as specific as possible to get better matches

Category *

Select a category

Location

e.g., Westlands, Nairobi

Budget (KSh) *

2500

Set a fair price to attract quality minions

Post Task

Cancel

💡 Tips for posting tasks:

- Be clear and specific about what needs to be done
- Mention any tools or materials the minion should bring
- Set a realistic budget based on the task complexity
- Include your availability or preferred time

My Tasks

My Tasks

Manage your posted tasks and review applications

Open

0

Pending

1

Assigned

0

Completed

0

Paid

1

All (2)

Open (0)

Pending (1)

Assigned (0)

Completed (0)

Paid (1)

Refresh

PENDING

Other

View Applications

Catering for a Family Event

Cook food enough for 13 people.

Location: Kabiria Posted: 3/20/2026

KSh 10,000

PAID

Other

Payment completed

Catering services for a wedding

I am looking for a competent chef to take the lead in the feeding of 100 guests at a wedding.

Location: Kinoo Posted: 2/16/2026

KSh 15,000

Payment History

Receipt: MANUAL-TASK-1-1773991541541
Amount: KSh 15,000
Paid Time: 3/20/2026, 10:25:41 AM

Rate this minion's work

★★★★★

No rating yet. Tap a star to rate.

Browse Minions

 **Browse Minions**

Review registered minions, their services, and availability before posting work.

Search minions

Search by name, services, or location...

Availability

All

Showing 2 of 2 registered minions

Refresh

Paul Munyua

Nairobi

flexible

Services

Repairs

CLIENT RATING

☆☆☆☆☆ 0.0 / 5 (0)

Experience: I do home repairs and makeovers

Phone: +25471

Email: @gmail.com

Joined 3/30/2026

View Profile

Post a Task

Awed Owl

Waithaka

flexible

Services

Cooking

CLIENT RATING

☆☆☆☆☆ 5.0 / 5 (1)

Experience: Professional Chef Offers outside catering services

Phone: +25471

Email: @gmail.com

Joined 2/16/2026

View Profile

Post a Task

Minion Dashboard

MINION DASHBOARD

AO

Welcome back, Awed!

Manage your work, stay visible to clients, and keep your earnings momentum moving forward.

Available

Logout

YOUR PROFILE

Awed Owl

A complete profile helps you match faster, stand out to clients, and win more suitable jobs.

MINION

Cooking

EMAIL

awedOwl21@gmail.com

PHONE

+254716982237

LOCATION

Waithaka

AVAILABILITY

flexible

Edit Profile

PROFILE STRENGTH

80% complete

Better profiles lead to better trust, stronger client responses, and more relevant task recommendations.

Phone number added

Location added

Skills listed

QUICK ACTIONS

Keep your workflow moving

Everything important is grouped into action cards so you can jump straight into work, updates, or new opportunities.

Find Gigs

Browse fresh opportunities that fit your location, skills, and availability.

Open →

My Accepted Tasks

Track assignments, update progress, and stay on top of client expectations.

Open →

Notifications

Check important task, payment, and platform updates in one place.

Open →

ACTIVE JOBS

COMPLETED

TOTAL EARNED

Find Gigs

Find Gigs

Browse available tasks and start earning today!

Search Tasks

Search by title, description, or location...

Category

All

Showing 1 of 1 tasks

Refresh

Repairs

KSh 34,000

Bedroom refurb

I want to switch up my bedroom from the paint on my walls to the tiling. Are you up for the task?

Kilimani, Nairobi

Posted by: Passion Berry

3/30/2026

Apply for Task

38

My Jobs

My Jobs

Track your applications and manage your accepted tasks

Pending

1

Accepted

2

Rejected

0

All (3)

Pending (1)

Accepted (2)

Rejected (0)

Refresh

Accepted

Repairs

Start Task

Bedroom refurb

I want to switch up my bedroom from the paint on my walls to the tiling. Are you up for the task?

Location: Kilimani, Nairobi

Client: Passion Berry

Applied: 3/30/2026

KSh 34,000

Your message: I am up for the task

Pending

Other

Waiting for client response...

Catering for a Family Event

Cook food enough for 13 people.

Location: Kabiria

Client: Passion Berry

Applied: 3/20/2026

KSh 10,000

Your message: great

Notifications pages

39

Notification Center

Review your latest action updates across registration, tasks, acceptance, and completion.

Total: 3 notifications

Mark All Read

Clear History

Application Accepted
Minion assigned successfully.
3/30/2026, 9:30:57 PM

Task Posted
Your task is live. Minions can now apply.
3/30/2026, 9:29:45 PM

Rating Saved
Thanks for your feedback.
3/30/2026, 9:25:10 PM

Sample of Notifications for Clients

Notification Center

Review your latest action updates across registration, tasks, acceptance, and completion.

Total: 2 notifications

Mark All Read

Clear History

Status Updated
Task marked as completed.
3/30/2026, 9:34:22 PM

Status Updated
Task marked as in progress.
3/30/2026, 9:34:19 PM

Sample of Notifications for Minions

Admin Dashboard

